



UNIVERSIDAD DE GRANADA



Tecnologías Web
Grado en Ingeniería Informática

Programación de aplicaciones en el servidor con PHP (3)
Gestión del estado de las aplicaciones

Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Ley 1/1996 de 12 de abril). Queda expresamente prohibido su uso o distribución sin autorización del autor.

(C) Javier Martinez Baena
jbaena@ugr.es
Departamento de Ciencias de la Computación e Inteligencia Artificial
<http://decsai.ugr.es>



Tecnologías Web
Grado en Ingeniería Informática

PHP(3) : Gestión del estado de las aplicaciones

» **1. Recordando el estado de las aplicaciones**

- Uso de GET, POST y campos hidden

2. Gestión de cookies

3. Gestión de sesiones

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martinez Baena



PHP: Gestión del estado de la aplicación
Introducción

Programación en la web

HTTP es un protocolo "stateless":

- Cada petición es independiente del resto (sin relación con peticiones previas).
- El servidor no recuerda peticiones previas

```
<?php
$x = $x + 3;
echo "X vale ", $x, PHP_EOL;
?>
```

Cada vez que se pide la página:

- PHP da un mensaje de error al usar \$x sin estar definida
- Imprime el mensaje X vale 3

¿Cómo recordar el estado entre solicitudes (ejecuciones)?

- Pasando variables por \$_GET en la URL
- Pasando variables ocultas en formularios (\$_POST + hidden)
- Cookies
- Variables de sesión
- Complementar métodos anteriores con almacenamiento de datos en BBDD en el servidor

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martinez Baena

3

PHP: Gestión del estado de la aplicación
Con variables GET/POST

Recordar estado usando formularios (\$_POST + hidden)

- Se incluyen controles de tipo *hidden* en formularios
- El paso de una página a otra se hace con controles "submit"

```

if (!isset($_POST['nombre']))
    $cuerpo = showFormName($_SERVER["SCRIPT_NAME"]);
else
    if (!isset($_POST['apellidos']))
        $cuerpo = showFormSurname($_SERVER["SCRIPT_NAME"], $_POST['nombre']);
    else
        if (!isset($_POST['trabajo']))
            $cuerpo = showFormWork($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos']);
        else
            $cuerpo = showData($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos'], $_POST['trabajo']);

```

Recordando Recordando Recordando Recordando

Introduce datos: Nombre: javier Enviar

Bienvenido Javier, continua ... Apellidos: Martínez Baena Enviar

Javier Martínez Baena, continua ... Trabaja en: Universidad de Granada Enviar

Datos recopilados: Nombre: Javier Apellidos: Martínez Baena Trabajo: Universidad de Granada

stateless01hidden

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 4

PHP: Gestión del estado de la aplicación
Con variables GET/POST

Recordar estado usando formularios (\$_POST + hidden)

- Se incluyen controles de tipo *hidden* en formularios
- El paso de una página a otra se hace con controles "submit"

```

if (!isset($_POST['nombre']))
    $cuerpo = showFormName($_SERVER["SCRIPT_NAME"]);
else
    if (!isset($_POST['apellidos']))
        $cuerpo = showFormSurname($_SERVER["SCRIPT_NAME"], $_POST['nombre']);
    else
        if (!isset($_POST['trabajo']))
            $cuerpo = showFormWork($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos']);
        else
            $cuerpo = showData($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos'], $_POST['trabajo']);

```

Recordando datos

Introduce datos: Nombre: javier Enviar

```

function showFormName($action) {
    return <<<HTML
    <div>
    <h2>Introduce datos:</h2>
    <form action=$action method='POST'>
    <label>Nombre: <input type='text' name='nombre'></label>
    <input type='submit' name='submit' value='Enviar'>
    </form>
    </div>
    HTML;
}

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 5

PHP: Gestión del estado de la aplicación
Con variables GET/POST

Recordar estado usando formularios (\$_POST + hidden)

- Se incluyen controles de tipo *hidden* en formularios
- El paso de una página a otra se hace con controles "submit"

```

if (!isset($_POST['nombre']))
    $cuerpo = showFormName($_SERVER["SCRIPT_NAME"]);
else
    if (!isset($_POST['apellidos']))
        $cuerpo = showFormSurname($_SERVER["SCRIPT_NAME"], $_POST['nombre']);
    else
        if (!isset($_POST['trabajo']))
            $cuerpo = showFormWork($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos']);
        else
            $cuerpo = showData($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos'], $_POST['trabajo']);

```

Recordando Recordando

Introduce datos: Nombre: javier Enviar

Bienvenido Javier, continua ... Apellidos: Martínez Baena Enviar

```

function showFormSurname($action, $nombre) {
    return <<<HTML
    <div>
    <h2>Introduce datos:</h2>
    <p>Bienvenido $nombre, continua ...</p>
    <form action=$action method='POST'>
    <label>Apellidos: <input type='text' name='apellidos'></label>
    <input type='hidden' name='nombre' value='$nombre'>
    <input type='submit' name='submit' value='Enviar'>
    </form>
    </div>
    HTML;
}

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 6

PHP: Gestión del estado de la aplicación

Con variables GET/POST

Recordar estado usando formularios (\$_POST + hidden)

- Se incluyen controles de tipo *hidden* en formularios
- El paso de una página a otra se hace con controles "submit"

```

if (!isset($_POST['apellidos']))
    $cuerpo = showFormSurname($_SERVER["SCRIPT_NAME"], $_POST['nombre']);
else
    if (!isset($_POST['trabajo']))
        $cuerpo = showFormWork($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos']);
    else
        $cuerpo = showData($_SERVER["SCRIPT_NAME"], $_POST['nombre'], $_POST['apellidos']);

function showFormWork($action, $nombre, $apellidos) {
    return <<<HTML
    <div>
    <h2>Introduce datos:</h2>
    <p>$nombre $apellidos, continua ...</p>
    <form action=$action method='POST'>
    <label>Trabajas en: <input type='text' name='trabajo'></label>
    <input type='hidden' name='nombre' value='$nombre'>
    <input type='hidden' name='apellidos' value='$apellidos'>
    <input type='submit' name='submit' value='Enviar'>
    </form>
    </div>
    HTML;
}
    
```



PHP: Gestión del estado de la aplicación

Con variables GET/POST

Recordar estado usando variables GET

- Se pasa un ID en el *query string* (evitando el uso de formularios)

```

http://servidor.com/pagina.php?nombre=Javier
http://servidor.com/pagina.php?nombre=Javier&apellidos=Martinez
...
http://servidor.com/otrapagina.php?nombre=Javier&apellidos=Martinez
    
```

Poco fiable / fácilmente falseable

Combinar GET/POST con BBDD

- Se envía solo un ID
- Se mantiene el resto de la información en una BBDD

pagina.php: Comprueba el ID y eventualmente podría:

- Recuperar información de una BBDD sobre cosas que haya hecho antes
- Almacenar nueva información en la BBDD para recordar lo que está haciendo

Necesita una BBDD (complica la aplicación)

PHP: Gestión del estado de la aplicación

Ejemplo




Cookie Editor

- ✓ _ga
- ✓ _ga
- ✓ _ga_JZ29GQGT7
- ✓ _ga_T4HYRHQW7X
- ✓ _gat
- ✓ _gid
- ✓ addtl_consent
- ✓ AWSALB
- ✓ AWSALBCORS
- ✓ cto_bundle
- ✓ euconsent-v2
- ✓ PHPSESSID

- Primera visita: muestra un aspecto
- Cambiamos a versión anterior
- Siguientes visitas: muestran un aspecto distinto

¿Cómo recuerda el sitio web la versión que hemos elegido?



UNIVERSIDAD
DE GRANADA

Tecnologías Web

Grado en Ingeniería Informática

PHP(3) : Gestión del estado de las aplicaciones

1. Recordando el estado de las aplicaciones
 - Uso de GET, POST y campos hidden
- » 2. Gestión de cookies
3. Gestión de sesiones



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



DECSAI

PHP: Gestión de cookies

¿Qué son?

Cookies

Una *cookie* son unos pocos bits de información que el servidor envía al cliente y este los almacena en forma de cadena de caracteres.

Cliente



3 Almacenar visitado=23



- 1 <http://servidor.org/hola.php>
- 2 **Respuesta HTTP:**
Headers incluye Set-Cookie: visitado=23
Cuerpo: respuesta HTML
- 4 <http://servidor.org/otra.php>
- 5 **Respuesta HTTP:**
Cuerpo: respuesta HTML
- 6 <http://servidor.org/otramas.php>
- 7 **Respuesta HTTP:**
Cuerpo: respuesta HTML

Servidor Web



hola.php

```
<?php
  setcookie('visitado','23');
  // ... generar respuesta HTML
?>
```

otra.php

```
<?php
  echo $_COOKIE['visitado'];
  // ... generar respuesta HTML
?>
```

otramas.php

```
<?php
  echo $_COOKIE['visitado'];
  // ... generar respuesta HTML
?>
```



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



DECSAI

PHP: Gestión de cookies

Cómo se usan

Cookies

- En cada petición del cliente al servidor, le envía las *cookies* en el *header* HTTP automáticamente.
- En el servidor, las *cookies* recibidas se almacenan en una variable superglobal (\$_COOKIE).

```
$caduca = 5; // Tiempo de caducidad (en segundos)
if (isset($_COOKIE["galleta"]))
  $msg = "Hola, me has visitado en los últimos ".$caduca." segundos así que aun tengo guardada tu cookie.
  Su valor es: ".$_COOKIE["galleta"];
else {
  setcookie("galleta", "Chocolate", time()+$caduca);
  $msg = "Hace tiempo que no te veo ... acabo de crear una cookie y se la he enviado a tu navegador.
  La próxima vez que visites esta página podrás ver su contenido. Ten en cuenta que si tardas más
  de ".$caduca." segundos la cookie se perderá.";
}
```

Primer acceso:

Recordando datos (cookies)

Hace tiempo que no te veo ... acabo de crear una cookie y se la he enviado a tu navegador. La próxima vez que visites esta página podrás ver su contenido. Ten en cuenta que si tardas más de 5 segundos la cookie se perderá.

Segundo y siguientes accesos (en próximos 5 segundos):

Recordando datos (cookies)

Hola, me has visitado en los últimos 5 segundos así que aun tengo guardada tu cookie. Su valor es: Chocolate

cookies00



DECSAI

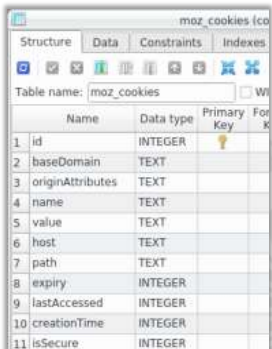
Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena

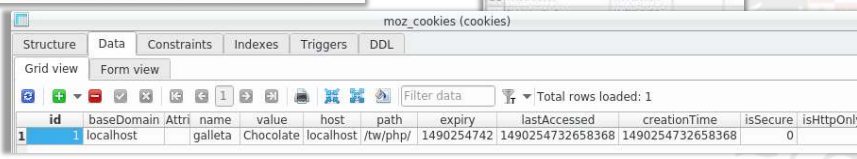


PHP: Gestión de cookies
Cómo se usan



Firefox (cliente) almacena la cookie en una BBDD de tipo SQLite:
~/.mozilla/Firefox/fk2z48tx.default/cookies.sqlite





Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

13



PHP: Gestión de cookies
Cómo se establecen

Cookies

```
setcookie(name,value,expire,path,domain,secure,httponly);
```

- name: identificador único de la cookie
- value: Valor almacenado (máximo 3-4KB)
- expire: (Opc) Fecha en la que caduca (se borra automáticamente).
Por defecto: 0 (caduca al cerrar el navegador)
- path: (Opc) La cookie está disponible en esa subcarpeta y en sus subcarpetas
Por defecto: carpeta actual
- domain: (Opc): La cookie está disponible para ese dominio y subdominios
Por defecto: dominio actual
- secure: La cookie solo está disponible en conexiones seguras (HTTPS)
Por defecto: false
- httponly: La cookie solo está disponible a través del protocolo HTTP (no es accesible por JavaScript).
Por defecto: false

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

14



PHP: Gestión de cookies
Cómo se borran

Eliminando cookies

La forma de eliminar una *cookie* es hacer que caduque.

Ejecución de `setcookie()` con idénticos parámetros que en la creación salvo por el tiempo de caducidad, que debe establecerse **en el pasado**

```
if (isset($_COOKIE["galleta"])) {
    echo "Tenemos una cookie almacenada: ", $_COOKIE["galleta"], PHP_EOL;
    setcookie("galleta", "Chocolate", time()-2592000);
    echo "... y la hemos borrado", PHP_EOL;
} else {
    setcookie("galleta", "Chocolate", time()+60);
    echo "... acabo de almacenar una cookie";
    echo "... y se borra en 60 segundos";
}
```

2592000 = 1 mes ... por si el reloj del cliente no está bien

Asignar la cadena vacía a una *cookie* también hace que se borre en el cliente

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

15

PHP: Gestión de cookies
Ejemplo: contador de visitas

Contador de visitas

cookies02contador

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 16

PHP: Gestión de cookies
Ejemplo: contador de visitas

Contador de visitas

```
$caducidad = 60*60*24; // Tiempo de caducidad de las cookies
if (isset($_POST['borrar'])) { // Se pulsó el botón de borrar cookie
    setcookie("visitas",'0',time()-1000); // Caducar cookie
    $numvisita = 0;
} else {
    if (!isset($_COOKIE['visitas']) || isset($_POST['poneracero']))
        $numvisita = 1; // Primera visita o reseteo
    else
        $numvisita = $_COOKIE["visitas"] + 1;
    setcookie("visitas",$numvisita,time()+$caducidad);
}

if ($numvisita==1) {
    $msg = 'Bienvenido, esta es su primera visita';
    if (isset($_POST['poneracero']))
        $msg .= ' ... pero ya has estado antes, y lo sabes';
} else if ($numvisita>1)
    $msg = '¿Qué hay de nuevo? esta es su visita número '.$numvisita;
else // $numvisita==0
    $msg = 'Vale, ha borrado su contador de visitas';
```

Calcular el número de visita según: \$_COOKIE y si se ha pulsado alguno de los botones

Determinar mensaje a imprimir

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 17

PHP: Gestión de cookies
Inconvenientes y seguridad

Inconvenientes

- No todos los clientes aceptan *cookies*
- El usuario puede deshabilitar/borrar las *cookies*
- Máximo de 4KB por *cookie* (nombre+valor+fecha caducidad+...)
- (Al menos) un máximo de 50 *cookies* por dominio, configurable
- (Al menos) un máximo de 3000 *cookies* en total, configurable
- Si se superan esos máximos el cliente podría hacer caducar otras *cookies* más antiguas que aún no debían caducar

RFC 6265 (2011)
Existe nuevo draft pendiente de aprobación desde 2022

Seguridad

- Las *cookies* se almacenan en el cliente: pueden modificarse
- Recomendaciones de seguridad:
 - Usa siempre conexiones **HTTPS** verificadas para que las *cookies* viajen cifradas
 - Usa **path** y **domain** para restringir el acceso a la *cookie*
 - Usa **secure=true** para evitar que se transfieran *cookies* si se usa HTTP (en lugar de HTTPS)
 - Usa **httponly=true** para evitar el acceso a *cookies* desde JavaScript (contra ataques XSS)

Para qué usar cookies

- Información no sensible (personalización, idioma, tema, etc.)
- Cookie de sesión (lo vemos a continuación)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 18



UNIVERSIDAD
DE GRANADA

Tecnologías Web

Grado en Ingeniería Informática

PHP(3) : Gestión del estado de las aplicaciones

1. Recordando el estado de las aplicaciones
 - Uso de GET, POST y campos hidden
2. Gestión de cookies
3. Gestión de sesiones



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



DECSAI

PHP: Gestión de sesiones

Cómo se usan

Sesiones PHP

Permiten recordar el estado de la aplicación de forma muy sencilla:

- Cualquier *script* PHP llama a `session_start()` al comienzo
- Desde ese momento podemos considerar que el array `$_SESSION` es "global" a la ejecución de todos esos *scripts*.

session1.php

```

<?php
session_start();
$_SESSION["nombre"] = "Javier";
echo '<a href="session1_ver.php">Pulsa para ir a la otra página</a>';
?>
```

```

<?php
session_start();
echo "Nombre = ", $_SESSION["nombre"];
?>
```

session_start() se llama al comienzo, **antes de generar código HTML**




DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



DECSAI

PHP: Gestión de sesiones

¿Qué son?

Sesiones PHP

Permiten recordar el estado de la aplicación con el siguiente mecanismo:

- El servidor crea un ID de sesión (PHPSESSID)
- Se establece un mecanismo de intercambio de PHPSESSID entre cliente y servidor
- Se almacenan datos en el servidor asociados al ID de sesión

Cliente



5 Almacenar PHPSESSID=XXX



1 `http://servidor.org/hola.php`

4 Respuesta HTTP:
Headers incluye
`Set-Cookie: PHPSESSID=XXX`
Cuerpo: respuesta HTML

6 `http://servidor.org/otra.php`

8 Respuesta HTTP:
Cuerpo: respuesta HTML

Servidor Web



2 Crea fichero local:
`/var/.../sess_XXX`

`<?php
session_start();
$_SESSION['nombre']='Javier';
// ... generar página HTML
?>`

3 Almacena



7 Lee fichero local:
`/var/.../sess_XXX`
en `$_SESSION`

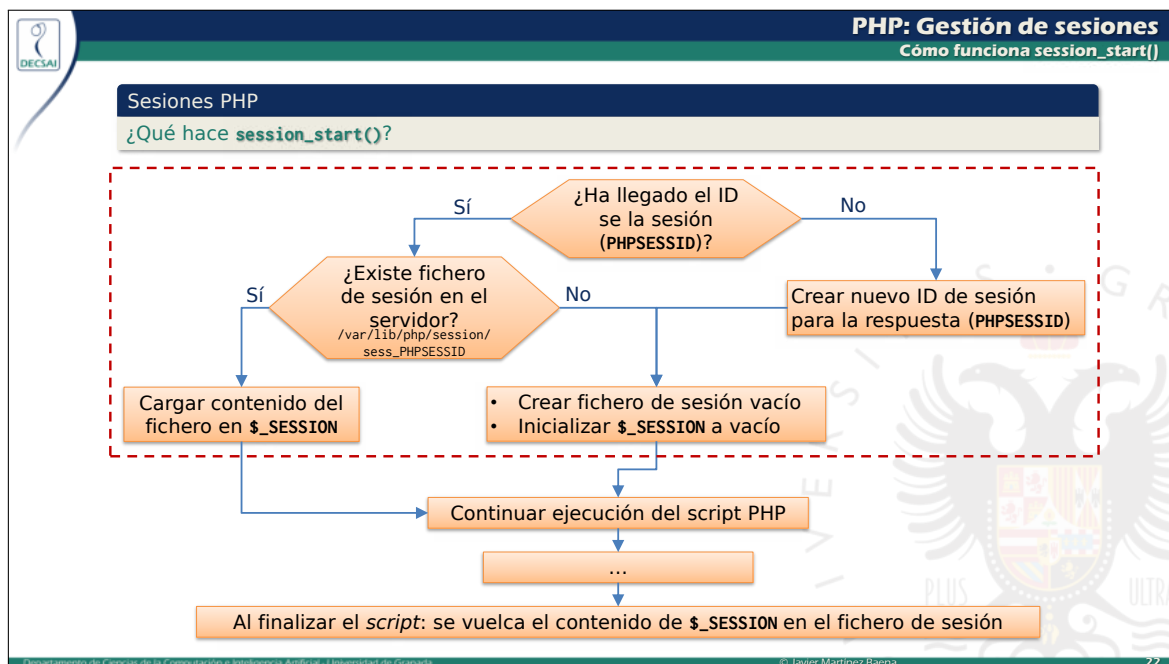
`<?php
session_start();
echo $_SESSION['nombre'];
// ... generar página HTML
?>`

XXX es de la forma vhahf8oh2b1kbaqdat435huckq



DECSAI

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



PHP: Gestión de sesiones

Ejemplo

Sesiones PHP

Ejemplo: recordar/olvidar valor desde un formulario

```

session_start();
require 'html.php';

if (isset($_POST['nombre'])) {
    $_SESSION['nombre'] = $_POST['nombre'];
    $cuerpo = showData($_SERVER["SCRIPT_NAME"], $_POST['nombre']);
} else if (isset($_POST['borrarsesion'])) {
    $cuerpo = showForm($_SERVER["SCRIPT_NAME"]);
} else if (isset($_SESSION['nombre'])) {
    $cuerpo = showData($_SERVER["SCRIPT_NAME"], $_SESSION['nombre']);
} else {
    $cuerpo = showForm($_SERVER["SCRIPT_NAME"]);
}
  
```

Recordando datos (sesiones)

Introduce datos:

Nombre:

[Pulsa aquí para cargar de nuevo esta página \(sin enviar el formulario\)](#)

Recordando datos (sesiones)

Bienvenido

Hola Javier. Ya me dijiste tu nombre en una ocasión anterior y me acordaré cada vez que me visites.

Si quieres que te olvide pulsa el botón (y borraré mi variable de sesión).

[Pulsa aquí para cargar de nuevo esta página \(sin enviar el formulario\)](#)

session00

23

PHP: Gestión de sesiones

Cómo y dónde se almacena la información de las sesiones

Sesiones PHP

Detalle de almacenamiento en cliente (cookie) y en servidor (sesión)

Cliente

Cookies

Domain	localhost
First-Party	
Name	PHPSESSID
Value	5hraj6rpqoa2aetf9f5na1gck9

El valor de la cookie es el ID de sesión

Servidor

```

<?php
session_start();
$_SESSION['nombre'] = "Javier";
...
?>
  
```

Sistema de archivos del servidor (un fichero por cada sesión):

/var/lib/php/sessions/sess_5hraj6rpqoa2aetf9f5na1gck9

nombre|s:6:"Javier";

El nombre del fichero incluye el valor del ID de sesión

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

24



PHP: Gestión de sesiones

Cómo se cierran

Finalizar una sesión

La función `session_destroy()` :

- Borra el fichero de almacenamiento de la sesión en el servidor
- (No destruye las variables de sesión (`$_SESSION`) por lo que se pueden seguir usando)
- (No elimina la cookie de la sesión (que el cliente sigue reenviando en cada petición))

La función `session_unset()` :

- Resetea el array `$_SESSION`
- (No destruye el fichero de sesión)
- (No elimina la cookie de sesión)

Recordando datos (sesiones)

La sesión se mantiene

Bienvenido

Hola Javier. Ya me dijiste tu nombre en una ocasión anterior y me acordaré cada vez que me visites. Si quieres que te olvide pulsa uno de los siguientes botones.

Observa la diferencia de comportamiento según el botón pulsado.
[Pulsa aquí para cargar de nuevo esta página \(sin enviar el formulario\)](#)

session01destroy

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

25



PHP: Gestión de sesiones

Cómo se cierran

Finalizar una sesión

Para hacer efectiva la destrucción de la sesión a todos los efectos es frecuente usar código similar a este:

```
function acabarSesion() {
    // La sesión debe estar iniciada
    if (session_status() == PHP_SESSION_NONE)
        session_start();

    // Borrar variables de sesión
    session_unset();

    // Obtener parámetros de cookie de sesión ...
    $param = session_get_cookie_params();
    // ... y borrar cookie de sesión
    setcookie(session_name(), $_COOKIE[session_name()], time()-2592000, $param['path'],
        $param['domain'], $param['secure'], $param['httponly']);

    // Destruir sesión
    session_destroy();
}
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

26



PHP: Gestión de sesiones

Sobre la configuración del servidor

Sesiones PHP

¿Y si el cliente tiene deshabilitadas las cookies?

Se puede pasar el ID de la sesión (PHPSESSID) ...

- ... por GET : En la URL
- ... por POST : campo hidden de formularios

Se puede hacer de forma manual o automática

```
ini_set("session.use_cookies", 0);
ini_set("session.use_only_cookies", 0);
ini_set("session.use_trans_sid", 1);
session_start();

session_start(['use_cookies' => '0',
    'use_only_cookies' => '0',
    'use_trans_sid' => '1']);
```

<code>use_cookies</code>	1 : utilizar cookies para propagar PHPSESSID 0 : PHPSESSID se envía en URL (parámetro GET)
<code>use_only_cookies</code>	1 : si el script recibe PHPSESSID a través de GET/POST → se descarta 0 : no descarta PHPSESSID proveniente de URL
<code>use_trans_id</code>	1 : activa modo SID transparente: los links y formularios se modifican (de forma automática) para pasar PHPSESSID 0 : desactiva modo SID transparente

Esto puede tener **problemas de seguridad** por lo que, en general, no se recomienda.



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

27



PHP: Gestión de sesiones
Sobre la configuración del servidor

Script PHP

```
function showForm($action) {  
    return <<<HTML  
        <div>  
            <h2>Introduce datos:</h2>  
            <form action=$action method='POST'>  
                <label>Nombre: <input type='text' name='nombre'></label>  
                <input type='submit' name='setnombre' value='Enviar'>  
            </form>  
            <p><a href="$action">Pulsa aquí para cargar de nuevo esta página (sin enviar el formulario)</a></p>  
        </div>  
    HTML;  
}
```

Código HTML generado por PHP

```
<div>  
<h2>Introduce datos:</h2>  
<form action=/tweb/tw2324/ejemplos/session02phpsessid/index.php method='POST'>  
    <input type="hidden" name="PHPSESSID" value="7d23skvsh8kirelfve51l200dj" />  
    <label>Nombre: <input type='text' name='nombre'></label>  
    <input type='submit' name='setnombre' value='Enviar'>  
</form>  
<p><a href="/tweb/tw2324/ejemplos/session02phpsessid/index.php?PHPSESSID=7d23skvsh8kirelfve51l200dj">Pulsa  
aquí para cargar de nuevo esta página (sin enviar el formulario)</a></p>  
</div>
```

Añadidos por PHP automáticamente

session02phpsessid

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

28